JDBC

What is JDBC and Why Do We Use It?

✅ JDBC (Java Database Connectivity):

- A **Java API** used to connect Java applications with relational databases (like MySQL, Oracle, PostgreSQL).
- It allows you to **insert, retrieve, update, and delete** data using SQL directly from Java.

🔍 Why JDBC is Needed:

> Java runs independently of databases. To talk to a database, Java needs a translator. That's JDBC.

📌 Example Use-Case:

Imagine you're building a **Student Management System**. You want to:

- Add student details to a database
- View all registered students

  👉 JDBC helps your Java code **send SQL queries** to the database and get back results.

🔷 2. JDBC Architecture

```
Java App → JDBC API → JDBC Driver → Database
```

- **Java App**: Your Java code.
- **JDBC API**: Standard interfaces (`Connection`, `Statement`, etc.)
- **JDBC Driver**: Provided by DB vendors (e.g., MySQL driver).

- **Database**: Where data is stored (e.g., MySQL, Oracle).

🔷 3. 5 Core Steps in JDBC

Step	What it Does	Why it's Important
1. Load Driver	Tells JVM which DB you're using	Without this, Java can't connect
2. Connect to DB	Opens connection to your DB	Needed for sending SQL
3. Create Statement	Prepares SQL commands	Sends commands
4. Execute Query	Executes SQL	Gets results
5. Close Connection	Frees resources	Avoid memory leaks

🔷 4. ✅ Setting Up JDBC with MySQL

🔧 Prerequisites:

- Install **MySQL** database
- Create a database:

```
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE users (id INT PRIMARY KEY AUTO_INCREMENT, name VARCH
AR(50), email VARCHAR(100));
```

📦 Add MySQL JDBC Driver:

If you're using Maven:

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
```

```
    <version>8.0.33</version>
</dependency>
```

🔷 5. 🔨 Full Code: JDBC Connection and Data Fetch

```
import java.sql.*;

public class Day1JDBC {
    public static void main(String[] args) {
        // JDBC URL syntax: jdbc:mysql://hostname:port/dbname
        String url = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "your_mysql_password";

        try {
            // 1. Load JDBC Driver (optional in latest versions)
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Establish the connection
            Connection con = DriverManager.getConnection(url, username, passw
ord);
            System.out.println("✅ Connected to DB successfully");

            // 3. Create Statement
            Statement stmt = con.createStatement();

            // 4. Execute Query
            ResultSet rs = stmt.executeQuery("SELECT * FROM users");

            // 5. Process ResultSet
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - " + rs.getString("name") + " -
```

```
" + rs.getString("email"));
            }

            // 6. Close connection
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

🔍 Code Explanation:

Code Line	Explanation
`Class.forName(...)`	Loads the MySQL JDBC driver
`DriverManager.getConnection(...)`	Opens a connection to the database
`Statement stmt = con.createStatement()`	Creates SQL execution object
`stmt.executeQuery(...)`	Runs the SQL SELECT
`rs.next()`	Moves through each row of the result
`con.close()`	Properly closes the DB connection

Insert, Update, Delete in JDBC (CRUD Operations)

✅ 1. Why Are These Operations Important?

Most real-world applications are dynamic — they **add, modify, or delete** data constantly. For example:

- A **student registration system** adds new students.
- An **admin portal** updates user info.
- A **delete account** button removes user records.

To do this, we need **INSERT**, **UPDATE**, and **DELETE** SQL commands via JDBC.

🔧 2. Prerequisites

Make sure you:

- Have a **MySQL database named** `testdb`
- Have a `users` **table**:

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(100)
);
```

🔨 3. Full Example Code: CRUD (Insert, Update, Delete)

```
import java.sql.*;

public class Day2JDBC_CRUD {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "your_mysql_password";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        Connection con = DriverManager.getConnection(url, username, passw
ord);

        // ✅ INSERT
        String insertSQL = "INSERT INTO users (name, email) VALUES (?, ?)";
        PreparedStatement insertStmt = con.prepareStatement(insertSQL);
        insertStmt.setString(1, "John Doe");
        insertStmt.setString(2, "john@example.com");
        int rowsInserted = insertStmt.executeUpdate();
        System.out.println("Rows inserted: " + rowsInserted);

        // ✅ UPDATE
        String updateSQL = "UPDATE users SET name = ? WHERE email = ?";
        PreparedStatement updateStmt = con.prepareStatement(updateSQL);
        updateStmt.setString(1, "Johnathan Doe");
        updateStmt.setString(2, "john@example.com");
        int rowsUpdated = updateStmt.executeUpdate();
        System.out.println("Rows updated: " + rowsUpdated);

        // ✅ DELETE
        String deleteSQL = "DELETE FROM users WHERE email = ?";
        PreparedStatement deleteStmt = con.prepareStatement(deleteSQL);
        deleteStmt.setString(1, "john@example.com");
        int rowsDeleted = deleteStmt.executeUpdate();
        System.out.println("Rows deleted: " + rowsDeleted);

        con.close();
      } catch (Exception e) {
        e.printStackTrace();
      }
   }
}
```

🔍 Code Breakdown

Part	What it Does
`PreparedStatement`	Used to run dynamic SQL safely (prevents SQL injection)
`setString(1, ...)`	Binds the first `?` in the SQL query
`executeUpdate()`	Used for `INSERT`, `UPDATE`, `DELETE` (returns number of rows affected)

⚠️ Why Use `PreparedStatement` Instead of `Statement`?

Statement	PreparedStatement
Less secure	Secure (prevents SQL injection)
Hard to maintain with dynamic data	Easy with `?` placeholders
Slower in repeated execution	Faster due to query pre-compilation

🔶 Introduction to Hibernate & ORM

🔶 What is ORM?

- **ORM** (Object-Relational Mapping) is a programming technique to map **Java classes to database tables**.
- Avoids manual SQL and ResultSet mapping.

🔶 Why Hibernate?

- Automates CRUD operations
- Handles relationships between tables
- Generates schema from Java code
- Provides caching and lazy loading

🔶 Hibernate Workflow:

```
Java Class (Entity) ←→ Hibernate ←→ SQL ←→ Database
```

🔶 Setting Up Hibernate

✅ Project Setup (No Spring Boot yet)

1. **Create a Maven project**
2. **Add Hibernate dependencies**:

```
!-- pom.xml -->
<dependencies>
  <!-- Hibernate Core -->
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.4.Final</version>
  </dependency>

  <!-- MySQL Driver -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
  </dependency>

  <!-- JPA API -->
  <dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.1.0</version>
  </dependency>
```

```
</dependencies>
```

1. **Create Hibernate Config:** `hibernate.cfg.xml`

```
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/hibernatedb</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">root</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>

    <mapping class="com.example.User"/>
  </session-factory>
</hibernate-configuration>
```

🔶First Hibernate Entity and CRUD

✅ Create Entity Class

```
package com.example;
```

```
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  @Column(name ="name" ,nullable = false)
  private String na;

  private String email;

  // Getters and setters
}
```

✅ Hibernate Utility Class

```
package com.example;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
  private static SessionFactory factory;

  static {
    factory = new Configuration().configure().buildSessionFactory();
  }

  public static SessionFactory getSessionFactory() {
    return factory;
```

```
  }
}
```

✅ Perform CRUD (Create Example)

```
package com.example;

import org.hibernate.Session;
import org.hibernate.Transaction;

public class Main {
  public static void main(String[] args) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx = session.beginTransaction();

    User user = new User();
    user.setName("Jay");
    user.setEmail("jay@example.com");

    session.save(user);

    tx.commit();
    session.close();
  }
}
```

🔶 Read, Update, Delete (CRUD continued)

✅ Read

```
User user = session.get(User.class, 1);
System.out.println(user.getName());
```

✅ Update

```
user.setEmail("newjay@example.com");
session.update(user);
```

✅ Delete

```
session.delete(user);
```

✅ Hibernate Annotations

🔶 Commonly Used JPA Annotations

Annotation	Purpose
`@Entity`	Marks a class as an entity (mapped to table)
`@Table(name = "tbl_name")`	Maps to specific table name
`@Id`	Marks primary key
`@GeneratedValue`	Defines primary key generation strategy
`@Column(name = "", ...)`	Maps field to column, add constraints
`@Transient`	Ignore this field (not mapped to DB)

✅ Example:

```
@Entity
@Table(name = "products")
public class Product {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  @Column(nullable = false)
  private String name;

  private double price;

  @Transient
  private String notSavedField;

  // Getters and Setters
}
```

🧪 **Practice**: Create a `Student` entity with `id`, `name`, `marks`, `grade`, and save a record.

✅ Primary Key Strategies

Hibernate supports several primary key generation strategies:

Strategy	Description
`AUTO`	Automatically chooses best option
`IDENTITY`	Uses auto-increment (MySQL, PostgreSQL)
`SEQUENCE`	Uses a database sequence (Oracle, PGSQL)
`TABLE`	Uses a table to simulate sequence

✅ Example:

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE)
private int id;
```

🧪 **Practice**: Try changing generation strategies and observe how DB reacts.

✅ HQL (Hibernate Query Language)

🔶 What is HQL?

- HQL is **object-oriented**, not table-oriented.
- It uses **entity names and fields**, not table/column names.

✅ Example:

```
List<Product> products = session.createQuery("from Product", Product.clas
s).list();
```

🔶 WHERE, ORDER BY, LIKE

```
// WHERE
List<Product> result = session.createQuery("from Product where price > :p",
Product.class)
                        .setParameter("p", 100.0)
                        .list();

// LIKE
List<Product> result = session.createQuery("from Product where name like :
```

```
n", Product.class)
                    .setParameter("n", "%Book%")
                    .list();
```

🧪 **Practice**: Query all products whose price is between 100 and 500.

✅ Native SQL Queries

Hibernate allows running **native SQL** when needed.

```
List<Object[]> rows = session.createNativeQuery("SELECT * FROM product
s").list();

for (Object[] row : rows) {
    System.out.println(row[0] + " - " + row[1]);
}
```

🧪 **Practice**: Use native SQL to get all products and print them manually.

✅ Named Queries

🔶 What is a Named Query?

Predefined query declared in Entity class using annotations.

✅ HQL Named Query

```
@Entity
@NamedQuery(
  name = "Product.findAllCheap",
  query = "from Product where price < :maxPrice"
)
public class Product {
```

```
    ...
}
```

✅ Using Named Query:

```
List<Product> cheapProducts = session.createNamedQuery("Product.findAll
Cheap", Product.class)
                              .setParameter("maxPrice", 200.0)
                              .list();
```

🧪 **Practice**: Create a named query to fetch all students with marks > 80.